Assignment 1

RAG: Baseline Retrieval vs. Re-Ranking

Subject: Language Models and Agents
Name: Priyanshu Panchasara
Roll Number: 2026701004
GitHub: `https://github.com/priyanshu-54/LMA_Assignment1`

1 Document and Setup

This experiment uses the *CFPB Small Entity Compliance Guide: Loan Originator Rule*, an 89-page document. The document was split into 198 chunks. Both retrieval methods used exactly the same chunks to generate Answer 1 (Baseline) and Answer 2 (Re-ranked) using `Qwen/Qwen2.5-3B-Instruct`.

2 Architecture Comparison

The core difference between the two models lies in how they process the query and document chunks:

- **Bi-Encoder (Baseline):** Uses independent embeddings for the query and the chunk, and calculates relevance using cosine similarity. This allows chunk vectors to be pre-computed and stored for fast retrieval, but misses nuanced relationships between query and text.
- **Cross-Encoder (Re-Ranking):** Employs joint self-attention across tokens, processing the query and the chunk together simultaneously. This allows for deep semantic comparison, but it is computationally expensive because it must be run dynamically for every (query, chunk) pair.

3 Qualitative & Rank Shift Analysis

The baseline method retrieved 10 chunks, sending the top 3 to the LLM (Answer 1). The re-ranking pipeline scored those 10 candidates with a Cross-Encoder, re-sorted them, and sent the new top 3 to the LLM (Answer 2).

Failure Case 1: Unexpected closing cost (Specific Exception)

- **Rank Shift:** The critical exception clause in Section 5.8 moved from Rank #2 (Baseline) to Rank #1 (Re-ranked).
- **Answer 1 vs. Answer 2:** While Answer 1 provided the general rule that commissions cannot fluctuate, Answer 2 successfully highlighted the specific exception (e.g., an unexpected title issue delaying closing) because the Cross-Encoder correctly prioritized the specific corner-case chunk over general definitions.

Failure Case 2: Negation & Constraints (Prohibited Actions)

- **Rank Shift:** A query asking if originators can receive compensation from both the consumer and the creditor saw the restrictive clause move from Rank #6 to Rank #2 after re-ranking.
- **Answer 1 vs. Answer 2:** Answer 1 generated a hallucination by stating it was partially allowed, as the baseline Bi-Encoder struggled with the negations and only retrieved positive-sounding context in the top 3. Answer 2 correctly stated the dual-compensation prohibition because the essential limiting clause was promoted into the top 3.

Keyword Trap Observation: For the query "How long do I need to keep my loan originator compensation records?", both methods kept the correct passage (Section 11.2) at Rank #1, successfully avoiding the heavily keyword-loaded distractor passage in Section 11.1.

Limitation: If a chunk is entirely missing from the baseline's initial Top 10 (e.g., dropping to Rank #64), the Cross-Encoder cannot promote it, demonstrating that re-ranking is fundamentally bottlenecked by the initial retrieval's recall.

4 Latency Profiling

The retrieval timings below outline the runtime of the baseline retrieval versus the re-ranking pipeline. Re-ranking adds substantial latency to the retrieval step itself, though answer generation remains the heaviest component.

Stage	Baseline (Answer 1)	Re-ranked (Answer 2)
Initial retrieval	17.8 ms	17.8 ms
Scoring 10 chunks with Cross-Encoder	–	292.3 ms
Total retrieval time	17.8 ms	310.1 ms
Answer generation (average)	51.27 s	58.51 s
Total time (average)	51.29 s	58.82 s

5 Scaling Question: Why Not Re-Rank Every Chunk?

We do not run a Cross-Encoder across the entire document collection from the start because of its massive computational cost. Because a Cross-Encoder uses joint self-attention on the query and chunk together, the embeddings cannot be pre-computed.

Scoring every single chunk in this 198-chunk document took approximately 6.7 seconds per query. If scaled to a real-world vector database containing one million chunks, running a Cross-Encoder across the entire collection would take nearly 9.4 hours for a single query. Conversely, a Bi-Encoder can search one million pre-computed vectors in roughly 124 ms. Therefore, the standard architecture uses the highly scalable Bi-Encoder to narrow the field to a small candidate pool (e.g., 10 to 50 chunks), and only applies the expensive Cross-Encoder to that narrowed list.